

Desarrollo de una Miniestación Meteorológica construida con Sensores y Materiales Reciclables, mediante el Paradigma de Programación Orientada a Objetos (POO)

Emanuel J. Torres M., Giomar O. Moreno Q., Loren S. Patarroyo P., Walexis S. Escalante V.

Universidad de Pamplona

Facultad de Ingenierías y Arquitecturas

Fundamentos de Programación Orientada a Objetos – 168411

Docente: Jesus Yndriago

19 de junio de 2026

Resumen—El presente informe abarca el diseño e implementación de una miniestación meteorológica automatizada programada en un microcontrolador ESP32, capaz de monitorear variables ambientales como temperatura, humedad relativa, humedad del suelo, iluminación y calidad del aire empleando los sensores electrónicos DHT11, HL-69, LDR y MQ-135; junto con la adición de una pantalla LCD, el módulo de comunicación I2C y un par de pulsadores para la visualización de datos en tiempo real.

El desarrollo del software del proyecto se realizó utilizando la metodología de diseño orientado a objetos, aplicando los conceptos y fundamentos propios de este paradigma de programación y desarrollando el software en Visual Studio Code mediante la extensión PlatformIO y el framework Arduino IDE, organizando el proyecto en un diseño modular con archivos de encabezado e implementación para mejorar la organización y el mantenimiento del código.

La parte física del proyecto tuvo en cuenta la creación de una placa de circuito impreso y reutilización de materiales reciclables y sobrantes de otros proyectos, minimizando el gasto económico y la huella ambiental de su producción.

Términos Clave—Estación Meteorológica, Arduino, Sensores, Programación Modular, Programación Orientada a Objetos (POO)

Resumen—This report covers the design and implementation of an automated mini-weather station programmed on an ESP32 microcontroller, capable of monitoring environmental variables such as temperature, relative humidity, soil moisture, light, and air quality using DHT11, HL-69, LDR, and MQ-135 electronic sensors; along with the addition of an LCD screen, an I2C communication module, and a pair of push buttons for real-time data display.

The project's software was developed using object-oriented design methodology, applying the concepts and fundamentals of this programming paradigm and developing the software in Visual Studio Code using the PlatformIO extension and the Arduino IDE framework, organizing the project into a modular design with header and implementation files to improve code organization and maintainability.

The hardware component of the project involved designing a printed circuit board and reusing recyclable materials and leftover parts from other projects, thereby minimizing both the financial cost and the environmental footprint of its production.

Términos Clave—Weather Station, Arduino, Sensors, Modular Programming, Object-Oriented Programming (OOP)

I. INTRODUCCIÓN

En la actualidad, el monitoreo de variables ambientales desempeña un papel fundamental en diferentes áreas como la agricultura, la industria y la investigación, ya que permite conocer las condiciones del entorno y tomar decisiones oportunas basadas en datos reales analizados. Parámetros como la temperatura, la humedad, la iluminación y la calidad del aire influyen directamente en diversos procesos de todo tipo, por lo que resulta importante contar con sistemas capaces de medirlos de forma continua y confiable.

Gracias al avance de los sistemas y de los microcontroladores, es posible desarrollar soluciones de bajo costo para la adquisición y visualización de información ambiental en tiempo real. Entre estas tecnologías se destaca el ESP32, una plataforma ampliamente utilizada por su capacidad de procesamiento, facilidad de programación y compatibilidad con una gran variedad de sensores electrónicos.

En este contexto, el presente informe abarca el desarrollo de una miniestación meteorológica automatizada capaz de monitorear variables ambientales como la temperatura y humedad relativa, la humedad del suelo, la iluminación y la calidad del aire mediante la implementación de un conjunto de sensores electrónicos capaces de leerlas, como el DHT11, HL-69, LDR y MQ-135, mientras que la información recolectada es procesada por el microcontrolador ESP32 y presentada al usuario mediante una pantalla LCD.

Además del desarrollo del sistema de monitoreo, el proyecto aplica los principios de la Programación Orientada a Objetos (POO), implementando conceptos como abstracción, encapsulamiento, herencia y polimorfismo para lograr una estructura de software modular, organizada y mantenible en el tiempo. De esta manera se realiza una integración de conocimientos en electrónica, programación y sistemas embebidos para el desarrollo de una solución funcional orientada al monitoreo de variables ambientales.

II. MARCO TEÓRICO

II-A. Estación Meteorológica

Una estación meteorológica se define formalmente como un emplazamiento técnico dotado de una variedad de instrumentos de medición, con el fin de observar, cuantificar, registrar y transmitir de manera sistemática los diferentes parámetros de las variables termodinámicas y cinéticas que configuran el estado físico del sistema atmosférico local [1].

En la ingeniería de sistemas embebidos se implementa la variante denominada Estación Meteorológica Automatizada. A diferencia de las estaciones analógicas convencionales que dependen enteramente de la lectura visual humana en intervalos temporales fijos, estas estaciones ejecutan de manera continua y autónoma los ciclos de muestreo, acondicionamiento de señal, almacenamiento digital y procesamiento analítico de datos climatológicos sin requerir intervención constante de operarios de campo [1].

La estructura lógica y funcional de una estación automatizada digital se compone de tres bloques arquitectónicos interconectados: un subsistema de transducción y adquisición, un subsistema central de procesamiento y un subsistema de interfaz y salida. El primero, constituido por una red de sensores electrónicos, transforma fenómenos físicos meteorológicos en señales eléctricas variables; el segundo, integrado por una unidad de microcontrolador, gestiona los tiempos de muestreo, ejecuta los algoritmos de filtrado digital y computa los promedios matemáticos requeridos para uniformizar los datos; y el tercero canaliza la información procesada hacia módulos de almacenamiento masivo o visualizadores locales en tiempo real [2], [3].

II-B. Programación Orientada a Objetos (POO)

POO es un paradigma de programación que utiliza representaciones de elementos de la vida real, llamados “objetos”, para resolver problemas y desarrollar aplicaciones informáticas. Dichos objetos poseen atributos (características) y métodos (acciones o comportamientos) propios que los definen [4].

La POO se diferencia de la programación estructurada cotidiana porque se fundamenta en cuatro pilares:

- **Herencia:** Permite que una clase hija herede características de otra clase padre, extendiendo la funcionalidad sin tener que reescribir el código principal, favoreciendo su reutilización.
- **Encapsulamiento:** Protege los datos que conforman una clase y controla su acceso a través de métodos específicos (*Setters* y *Getters*) definidos como públicos, protegidos o privados en función de su alcance.
- **Polimorfismo:** Permite que un mismo método tenga comportamientos diferentes dependiendo de la clase que lo utilice y los atributos que le sean entregados.
- **Abstracción:** Modela solo los comportamientos esenciales para el funcionamiento general de un objeto, omitiendo los detalles internos o complejos de funcionamiento.

En este paradigma los programas se construyen mediante la interacción de objetos, los cuales son instancias de clases;

mejorando la modularidad, la gestión de memoria, la organización y la reutilización del código a largo plazo [4], [5]. Una clase describe la esencia de un objeto, ya que es la abstracción que abarca los atributos, operaciones, relaciones y semántica comunes en un conjunto de objetos.

II-C. Programación Modular

Es un paradigma de desarrollo de software que consiste en dividir un programa grande en componentes o módulos independientes, cada uno encargado de realizar una función específica dentro del sistema. Esta estructura permite organizar el código de manera más estructurada, facilitando su comprensión, mantenimiento y reutilización. Cada módulo puede desarrollarse, probarse y modificarse de forma independiente [5], [6].

En el lenguaje C++, su implementación se realiza comúnmente mediante la separación del código en archivos de cabecera (.h) y archivos fuente o de implementación (.cpp). Los archivos de cabecera contienen las declaraciones de clases, funciones y métodos, mientras que los archivos de implementación almacenan la lógica y el comportamiento de dichos elementos [6], [7].

II-D. Microcontrolador

Una MCU (Unidad de Microcontrolador) es un dispositivo constituido por un único circuito integrado capaz de ejecutar múltiples tareas siguiendo una secuencia lógica de instrucciones programadas y de interactuar con su entorno mediante pines destinados a dispositivos periféricos de entrada y salida [8].

En comparación con los ordenadores, PLC u otros controladores programables, los microcontroladores destacan por su reducido tamaño, bajo consumo energético y bajo costo comercial. Su arquitectura flexible les permite ser utilizados en áreas como la robótica, la electrónica, la industria automotriz, el IoT, la biomédica y los sistemas de navegación espacial [3].

II-E. Arduino

Arduino es un software y hardware Open Source que ha democratizado el acceso al desarrollo de proyectos electrónicos y sistemas embebidos, permitiendo a usuarios analizar, modificar y distribuir recursos libremente [9].

Creada en 2005 por un equipo del Instituto de Diseño Interactivo de Ivrea, Arduino está conformado por una placa electrónica con periféricos de entrada y salida, un microcontrolador ATmega (generalmente ATmega328) y un entorno de desarrollo integrado (IDE) que permite escribir, compilar y cargar código al microcontrolador vía USB [9], [10].

Un complemento importante para el desarrollo de proyectos con esta MCU es Tinkercad. Desarrollada por Autodesk, esta plataforma en línea permite diseñar y simular circuitos electrónicos utilizando microcontroladores como Arduino y Microbit [11].

II-F. ESP32

El ESP32, desarrollado por Espressif Systems para aplicaciones de IoT, sistemas embebidos y dispositivos electrónicos de bajo consumo energético, incorpora conectividad Wi-Fi y Bluetooth, lo que le permite comunicarse con una gran variedad de dispositivos y módulos mediante interfaces como SPI e I2C [12], [13].

De acuerdo con su documentación oficial, el nivel de integración de los componentes facilita significativamente su incorporación en circuitos impresos (PCB) [12]. La arquitectura del ESP32 le permite ejecutar programas desarrollados en diferentes entornos, tanto Arduino IDE como el framework oficial ESP-IDF.

II-G. Sensores

Son dispositivos electrónicos diseñados para detectar cambios y estímulos en el entorno y transformarlos en señales digitales, las cuales son convertidas en impulsos eléctricos para que puedan ser leídas y procesadas por sistemas electrónicos y microcontroladores [14].

II-G1. Sensor de Humedad del Suelo (HL-69): El sensor capacitivo de humedad es un dispositivo utilizado para medir el nivel de humedad presente en un terreno mediante la variación de capacitancia producida por los cambios en el contenido de agua del suelo [15]. A diferencia de los sensores resistivos, presenta mayor resistencia a la corrosión y mejor estabilidad durante su funcionamiento.

II-G2. Sensor de Temperatura y Humedad Relativa (DHT-11): Es un sensor digital de bajo costo diseñado para medir temperatura y humedad relativa del ambiente, integrando un sensor capacitivo de humedad y un termistor, entregando datos mediante una señal digital [16]. Su facilidad de implementación lo hace compatible con microcontroladores como Arduino y ESP32.

II-G3. Sensor LDR: Está compuesto por una resistencia dependiente de luz cuyo valor resistivo varía en función de la intensidad lumínica percibida. A mayor cantidad de luz incidente, menor será el valor de su resistencia eléctrica [17].

II-G4. Sensor de Calidad del Aire (MQ-135): Es un sensor semiconductor utilizado para detectar diferentes gases peligrosos presentes en el aire, entre los cuales están el dióxido de carbono (CO_2), amoníaco (NH_3), benceno, humo y compuestos orgánicos volátiles (VOCs) [18].

II-H. Pantalla de Cristal Líquido (LCD)

Es un dispositivo de visualización optoelectrónico plano fundamentado en la manipulación controlada de las propiedades ópticas de los cristales líquidos en presencia de campos eléctricos. En una pantalla LCD nemática de torsión, al no aplicar voltaje las moléculas se alinean en una trayectoria helicoidal que rota la polarización de la luz incidente, permitiendo que atraviese el segundo polarizador y el píxel se observe iluminado. Al aplicar voltaje se destruye la estructura helicoidal, la luz es absorbida por el segundo filtro y el píxel toma apariencia oscura [20].

En sistemas embebidos de medición, los módulos LCD alfanuméricos monocromáticos de 16 columnas por 2 filas son uno de los estándares más comunes, con una matriz interna de 5×8 puntos por carácter. Para evitar que el microcontrolador deba gestionar individualmente la conmutación eléctrica, la pantalla integra el controlador HD44780.

II-I. Módulo de Comunicación I2C

El bus I2C es un protocolo de comunicación serie, síncrono, semidúplex y orientado a bytes, desarrollado por NXP Semiconductors [19]. Su concepción responde a la necesidad de intercomunicar múltiples circuitos integrados dentro de una misma tarjeta de circuito impreso, optimizando el espacio físico y reduciendo la complejidad de este.

El bus se compone de dos líneas físicas: SDA (transmisión secuencial de datos) y SCL (pulsos de reloj). A nivel de hardware, ambas líneas están configuradas con etapas de salida de colector abierto y se requieren dos resistencias de elevación para garantizar los niveles lógicos por defecto [19].

La arquitectura operativa de I2C se basa en un esquema jerárquico de maestro y esclavo, donde el dispositivo maestro posee el control absoluto del bus, siendo el único facultado para iniciar las transferencias de datos [16], [17].

II-J. Repositorio en Línea GitHub

GitHub es una plataforma de desarrollo colaborativo basada en la nube creada por Microsoft, que permite almacenar, gestionar y compartir proyectos de software mediante el sistema de control de versiones Git. Incorpora herramientas para la gestión de ramas, revisión de código, seguimiento de errores y documentación de proyectos.

II-K. Sistema de Control de Versiones (Git)

Git es un sistema de control de versiones de código abierto diseñado para registrar cronológicamente los cambios realizados en el código fuente de un proyecto [21]. Su flujo de trabajo se organiza en tres secciones lógicas: el directorio de trabajo, el área de preparación (*staging area*) y el repositorio local. El uso de Git resulta fundamental para gestionar de manera segura el desarrollo de software de sistemas embebidos y proyectos de gran escala [22].

III. METODOLOGÍA

El desarrollo de la Miniestación Meteorológica Automatizada se llevó a cabo mediante fases de diseño e implementación por etapas, fundamentada en los principios de POO y la integración coherente entre el hardware y el software del sistema.

Como punto de partida, se realizó un análisis exhaustivo de los requerimientos del sistema, en el cual se identificaron 5 variables ambientales objeto de monitoreo: temperatura y humedad ambiental, humedad del suelo, intensidad lumínica y calidad del aire, así como el microcontrolador encargado de la lectura de las mediciones tomadas por los dispositivos de monitoreo.

A partir de esta necesidad, se seleccionó un microcontrolador ESP32 DevKit-V1, así como los sensores DHT11, HL-69,

LDR y MQ-135 por su asequibilidad y sencilla programación, y una pantalla LCD junto a un módulo I2C para la visualización local de los datos recolectados en tiempo real.

Una vez definidos los requerimientos, se realizó el diseño estructural del sistema mediante la elaboración de diagramas UML que evidenciaran la incorporación de POO en el proyecto. Para ello, se estableció una clase abstracta denominada *Sensor* con el propósito de definir una interfaz común para todos los dispositivos de medición, una clase particular para la pantalla LCD y una clase maestra *EstacionMeteoreologica* encargada de la gestión conjunta de todos los sensores, lecturas, toma de datos e impresiones.

A partir de la clase *Sensor* se implementaron las clases derivadas *SensorAmbiental*, *SensorSuelo*, *SensorLuz* y *SensorAire* con sus respectivos métodos y constructores sobrescritos haciendo uso de los mecanismos de herencia y polimorfismo propios de POO.

La fase de implementación utilizó Visual Studio Code con la extensión PlatformIO y el framework de Arduino IDE. El proyecto se organizó en archivos de encabezado (.h) y archivos de implementación (.cpp), y se vinculó a un repositorio web en GitHub con control de versiones a través de Git, de manera que se pudieran contrastar los cambios en el tiempo y crear nuevas ramas (*branches*) en caso de cambios experimentales.

Posteriormente, se realizó la integración física del hardware a través de una placa de circuito impreso (PCB). Tras la integración de cada sensor y la pantalla al circuito del ESP32, se adicionaron 2 pulsadores normalmente abiertos que permitirían el desplazamiento de la información mostrada por la LCD a través de un *array*.

Para complementar la presentación del proyecto, se preparó una maqueta física compuesta en su mayoría por material reciclable o elementos sobrantes de proyectos anteriores, como la base en impresión 3D, una tabla para el techo y soportes hechos de escoba.

Finalmente, se llevó a cabo una etapa de pruebas y validación que comprendió pruebas individuales de cada dispositivo y pruebas de integración orientadas a comprobar la correcta interacción entre los distintos componentes del sistema, evaluando la lectura de datos, las actualizaciones en la pantalla LCD y el comportamiento del programa bajo diversas condiciones ambientales.

IV. ANÁLISIS DE RESULTADOS

Durante la fase de testeo, las pruebas realizadas indicaron un correcto funcionamiento de los sensores implementados y comunicación entre el microcontrolador y la pantalla de visualización en la miniestación meteorológica.

El sensor DHT11 permitió obtener lecturas de temperatura y humedad relativas cercanas a las condiciones del entorno, mostrando una respuesta estable y consistente durante las pruebas, teniendo en cuenta su condición como un sensor exclusivamente digital poco preciso.

Análogamente, el sensor HL-69 detectó variaciones en la humedad del suelo en tiempo real al entrar en contacto con

los diferentes niveles de humedad a los que fue expuesto, permitiendo identificar valores intermedios entre el suelo seco y húmedo y picos de humedad.

Al contrario, respecto al sensor de luz (LDR), se observó que las lecturas presentaban únicamente dos estados posibles: Ausencia de luz o iluminación máxima. Este comportamiento se debe principalmente a que la referencia adquirida solo cuenta con una salida digital (D0) en vez de una salida analógica, siendo imposible medir la intensidad luminosa de forma gradual, aunque sigue siendo útil para detectar cambios drásticos en la iluminación del ambiente.

Por otra parte, el sensor MQ-135 respondió satisfactoriamente a las variaciones de partes por millón (PPM) producto de la acción de agentes contaminantes como el humo, mostrando lecturas analógicas coherentes. Si bien los valores obtenidos no representan directamente una concentración específica de gases, permitieron evidenciar la capacidad del sistema para monitorear condiciones ambientales relacionadas con la calidad del aire.

Asimismo, las pruebas individuales realizadas para la pantalla LCD mostraron la información obtenida por los sensores en tiempo real, así como durante las pruebas de integración donde se realizaron impresiones a intervalos de tiempo regulares y la modificación de la información al interactuar con pulsadores.

La implementación de POO mediante clases, herencia, encapsulamiento y polimorfismo permitió desarrollar un código modular, organizado y fácil de editar y mantener incluso ante la incorporación de nuevos sensores o dispositivos al sistema.

En general, los resultados obtenidos demostraron que la miniestación meteorológica fue capaz de adquirir, procesar y visualizar la mayor parte de la información ambiental de manera satisfactoria; sin embargo, se identificaron el uso de un sensor LDR con salida analógica (A0), una calibración más precisa del sensor MQ-135 y el cambio del sensor DHT11 por el DHT22 como puntos de mejora para obtener mediciones más exactas y coherentes con las variaciones ambientales.

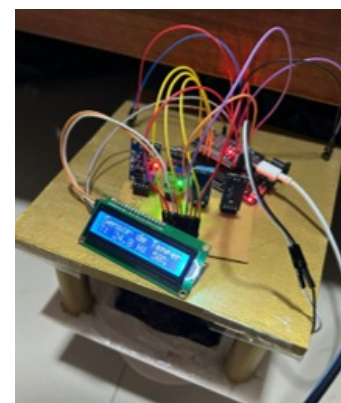


Figura 1. Prototipo de la Miniestación Meteorológica funcional en la maqueta física mostrando lecturas del *SensorAmbiental* (temperatura y humedad relativa).



Figura 2. Mediciones de *SensorAmbiental* y *SensorAire* de la Miniestación Meteorológica durante una prueba de integración.

V. CONCLUSIONES

El desarrollo de la miniestación meteorológica automatizada permitió monitorear variables ambientales y sus fluctuaciones en el tiempo satisfactoriamente, a través de la integración de sensores de temperatura y humedad relativa, humedad del suelo, iluminación y calidad del aire soportados por el microcontrolador ESP32, así como mostrar dichas lecturas visualmente en tiempo real a través de una pantalla LCD, facilitando la supervisión localmente.

Asimismo, la implementación de la Programación Orientada a Objetos contribuyó significativamente a la organización y estructuración del software, permitiendo aplicar conceptos como abstracción, encapsulamiento, herencia y polimorfismo en la construcción de este, lo cual favoreció la modularidad del código, su mantenimiento, comprensión y posible ampliación para futuras versiones del proyecto.

Por su parte, la combinación del microcontrolador ESP32 con el uso de materiales reciclables para la construcción de la maqueta física dio como resultado un prototipo que combina eficiencia, bajo costo y sostenibilidad. Este diseño demuestra que la reutilización de componentes y estructuras es clave para elaborar soluciones tecnológicas funcionales que minimizan el gasto presupuestario y la huella ambiental.

No obstante, las pruebas realizadas permitieron identificar las capacidades y limitaciones de los sensores utilizados. En particular, el sensor LDR únicamente permitió detectar estados de presencia o ausencia de luz, mientras que el DHT11 proporcionó una referencia general de temperatura y humedad, más careció de precisión. Esto evidencia la importancia de la correcta selección y configuración de los dispositivos de medición para obtener resultados más precisos.

Finalmente, el proyecto permitió fortalecer los conocimientos adquiridos en electrónica analógica, programación de microcontroladores y diseño de proyectos orientado a objetos, integrando componentes de hardware y software en una solución tecnológica funcional y asequible, que puede mejorarse a través de la implementación de procesos de calibración más avanzados, el uso de sensores con mayor precisión y la incorporación de sistemas de almacenamiento o transmisión de datos de manera inalámbrica.

REFERENCIAS

- [1] Organización Meteorológica Mundial, *Guía de instrumentos y métodos de observación meteorológicos*, OMM-N.º 8, 7.ª ed. Ginebra, Suiza: Secretaría de la OMM, 2008.

- [2] T. L. Floyd, *Dispositivos electrónicos*, 8.ª ed. México D.F.: Pearson Educación, 2008, pp. 110–115.
- [3] T. L. Floyd, *Fundamentos de sistemas digitales*. [En línea]. Disponible en: https://gc.scalahed.com/recursos/files/r161r/w25733w/fundamentos-de-sistemas_compressed.pdf. [Accedido: 31-may-2026].
- [4] J. C. Moreno, “Programación Orientada a Objetos (MF0227_3)”, Spain: Rama Editorial, 2016. [En línea]. Disponible en: <https://www-digitaliapiublishing-com.unipamplona.basesdedatosezproxy.com/a/110026>. [Accedido: 31-may-2026].
- [5] P. J. Ponce de León Amador y E. Saquete Boro, “Introducción a la programación orientada a objetos”, Spain: Publicaciones de la Universidad de Alicante, 2006. [En línea]. Disponible en: <https://www-digitaliapiublishing-com.unipamplona.basesdedatosezproxy.com/a/701>. [Accedido: 31-may-2026].
- [6] P. J. Deitel y H. M. Deitel, *C++: Cómo programar*, 10.ª ed. Ciudad de México, México: Pearson Educación.
- [7] B. Stroustrup, *The C++ Programming Language*, 4th ed. Boston, MA, EE. UU.: Addison-Wesley Professional.
- [8] N. F. Penagos Quintero, “Microcontroladores: Microchip, Atmel, NXP Freescale y Texas Instruments. Pasos para una programación con éxito”, *Universidad Piloto de Colombia*, 2022. [En línea]. Disponible en: <https://www-digitaliapiublishing-com.unipamplona.basesdedatosezproxy.com/a/119483>. [Accedido: 31-may-2026].
- [9] M. A. Salazar Guilcamaigua y P. A. Portero Donoso, “Introducción a la programación con Arduino: Guía básica con ejemplos prácticos”, Editorial Exced, 2025. [En línea]. Disponible en: <https://www-digitaliapiublishing-com.unipamplona.basesdedatosezproxy.com/a/180237>. [Accedido: 31-may-2026].
- [10] E. López, “Arduino. Guía práctica de fundamentos y simulación”, Rama Editorial, 2016. [En línea]. Disponible en: <https://www-digitaliapiublishing-com.unipamplona.basesdedatosezproxy.com/a/110057>. [Accedido: 31-may-2026].
- [11] Autodesk, “Basics of Arduino – Tinkercad”, *Tinkercad*. [En línea]. Disponible en: <https://www.tinkercad.com/projects/Basics-of-Arduino-TINKERCAD>. [Accedido: 31-may-2026].
- [12] Espressif Systems, “ESP32 Series”, *Espressif Systems*. [En línea]. Disponible en: <https://www.espressif.com/en/products/socs/esp32>. [Accedido: 31-may-2026].
- [13] Espressif Systems, “ESP32 Wi-Fi & Bluetooth Modules”, *Espressif Systems*. [En línea]. Disponible en: <https://www.espressif.com/en/products/modules/esp32>. [Accedido: 31-may-2026].
- [14] SDI Industrial, “Sensores: definición y funcionamiento”, *SDI Industrial*. [En línea]. Disponible en: <https://sdiindustrial.com.mx/blog/sensores/>. [Accedido: 31-may-2026].
- [15] EOS Data Analytics, “Sensores de humedad del suelo”, *EOSDA*. [En línea]. Disponible en: <https://eos.com/es/blog/sensores-de-humedad-del-suelo/>. [Accedido: 31-may-2026].
- [16] Naylamp Mechatronics, “Sensor de temperatura y humedad relativa DHT11”, *Naylamp Mechatronics*. [En línea]. Disponible en: <https://naylampmechatronics.com/sensores-temperatura-y-humedad/57-sensor-de-temperatura-y-humedad-relativa-dht11.html>. [Accedido: 31-may-2026].
- [17] ALLELCO, “¿Qué es un LDR y cómo funciona en electrónica?”, *ALLELCO Electronics*. [En línea]. Disponible en: <https://www.allelcoelec.es/blog/what-is-an-ldr-and-how-it-works-in-electronics>. [Accedido: 31-may-2026].
- [18] Instructables, “Calidad de aire – Sensor MQ-135”, *Instructables*. [En línea]. Disponible en: <https://www.instructables.com/Calidad-De-Air-Sensor-Mq-135/>. [Accedido: 31-may-2026].
- [19] NXP Semiconductors, “I2C-bus specification and user manual”, Rev. 6, Doc. UM10204, abr. 2014. [En línea]. Disponible en: <https://www.nxp.com/docs/en/user-manual/UM10204.pdf>. [Accedido: 31-may-2026].
- [20] Texas Instruments, “PCF8574 Remote 8-Bit I/O Expander for I2C Bus”, Hoja de datos SCPS068J, Dallas, TX, EE. UU.
- [21] S. Chacon y B. Straub, “Pro Git”, 2.ª ed. Nueva York, EE. UU.: Apress. [En línea]. Disponible en: <https://git-scm.com/book/es/v2>. [Accedido: 31-may-2026].
- [22] E. Ribeiro, *Git for Embedded System Developers: Version Control for Hardware and Firmware*. Londres, Reino Unido: Packt Publishing, 2019, pp. 88–102.

ANEXOS

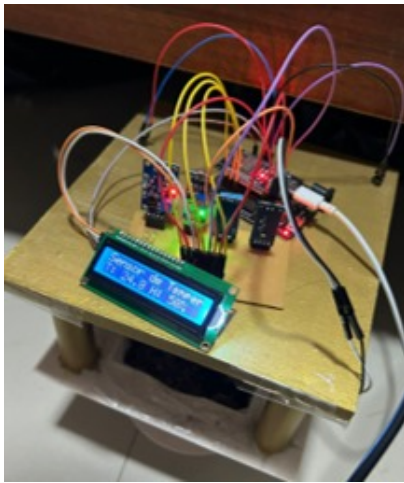


Figura 3. Prototipo 1 de la Miniestación Meteorológica montado sobre base de madera con pantalla LCD y sensores conectados.

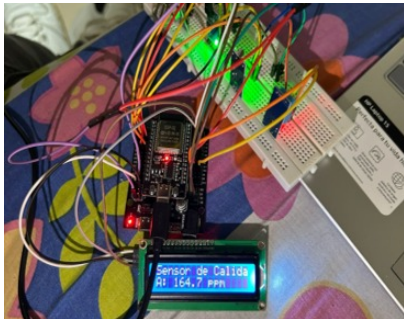


Figura 4. Desarrollo de conexiones en protoboard del segundo prototipo funcional. La pantalla LCD muestra lectura del sensor de calidad del aire (164.7 ppm).



Figura 5. Sensor MQ-135 para detección de calidad del aire (CO_2 , NH_3 , humo y VOCs).



Figura 6. Sensor HL-69 para medición de humedad del suelo.



Figura 7. Módulo sensor LDR para medición de nivel de iluminación ambiental.

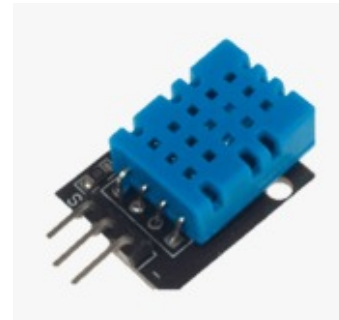


Figura 8. Sensor DHT11 para medición de temperatura y humedad relativa del ambiente.



Figura 9. Pantalla LCD mostrando lectura en tiempo real del *SensorCalidad*: 164.7 ppm de calidad del aire.



Figura 10. Pantalla LCD mostrando lectura en tiempo real del *SensorAmbiental*: temperatura 24.4 °C y humedad relativa 50%.

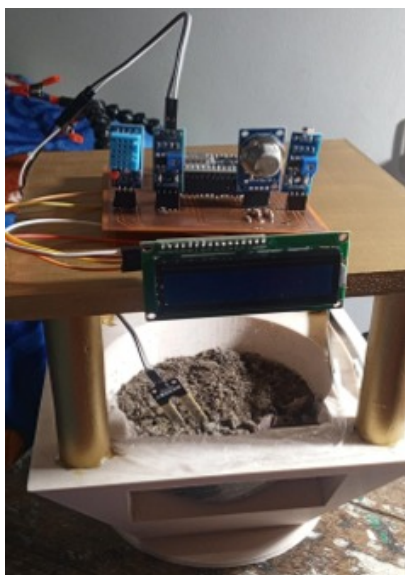


Figura 11. Maqueta física final de la Miniestación Meteorológica con el sensor HL-69 insertado en sustrato de suelo para prueba de humedad.

Cuadro I
FICHA TÉCNICA DEL SENSOR DHT-11

Característica	Valor
Tipo de sensor	Temperatura y humedad relativa
Voltaje de operación	3.3 V – 5 V DC
Corriente de operación	0.3 mA (medición)
Corriente en espera	60 μ A
Potencia aproximada	1.5 mW
Rango de temperatura	0 °C a 50 °C
Precisión temperatura	± 2 °C
Rango de humedad	20 % a 90 % HR
Precisión humedad	± 5 % HR
Tiempo de respuesta	Menor a 6 segundos
Tipo de salida	Digital

Cuadro II
FICHA TÉCNICA DEL SENSOR LDR

Característica	Valor
Tipo de sensor	Resistencia dependiente de la luz
Variable medida	Intensidad lumínica
Voltaje de operación	Depende del circuito
Corriente de operación	Muy baja (μ A a mA)
Potencia aproximada	Menor a 100 mW
Tipo de salida	Analógica
Resistencia en oscuridad	Hasta varios M Ω
Resistencia con luz intensa	Cientos de Ω a pocos k Ω
Tiempo de respuesta	20 ms – 100 ms
Ángulo de detección	120° – 180°

Figura 12. Desarrollo de las conexiones en protoboard para el primer prototipo de la Miniestación Meteorológica.

Figura 13. Prototipo funcional de la Miniestación Meteorológica en la maqueta física mostrando lecturas del *SensorAmbiental* (temperatura y humedad relativa).

Figura 14. Código QR de acceso al repositorio de GitHub del proyecto.

Cuadro III
FICHA TÉCNICA DEL SENSOR HC-69

Característica	Descripción
Tipo de sensor	Sensor capacitivo de humedad del suelo
Variable medida	Humedad del suelo
Voltaje de operación	3.3 V a 5.5 V DC
Corriente de operación	Menor a 5 mA
Potencia aproximada	Menor a 25 mW
Tipo de salida	Analógica
Principio de funcionamiento	Variación de la capacitancia según el contenido de agua del suelo
Rango de detección	Desde suelo seco hasta suelo completamente húmedo
Precisión	Depende de la calibración y del tipo de suelo
Material sensible	Placa capacitiva anticorrosión
Resistencia a la corrosión	Alta
Tiempo de respuesta	Rápido, en tiempo real
Ventajas principales	Mayor durabilidad, estabilidad y resistencia a la corrosión en comparación con sensores resistivos

Cuadro IV
FICHA TÉCNICA DEL SENSOR MQ-135

Característica	Valor
Tipo de sensor	Calidad del aire
Voltaje de operación	5 V DC
Corriente de operación	≈ 150 mA
Potencia del calentador	≈ 800 mW
Tipo de salida	Analógica y digital
Gases detectados	CO ₂ , NH ₃ , H ₂ S, benceno, alcohol, humo y VOCs
Rango de detección	10 ppm – 1000 ppm
Temperatura de operación	-10 °C a 50 °C
Humedad de operación	Hasta 95 % HR
Tiempo de precalentamiento	24 – 48 h
Sensibilidad	Ajustable mediante calibración